

CSE 546: Applied Cryptography - 2026

Mid-Semester Exam

23-February-2026

Duration: 2 hrs

Total: 50 marks

Name: _____

Roll No: _____

Instructions:

1. You are required to attempt **ALL** the 10 questions. Each question carries **5 marks**.
2. A bonus of **2 marks** will be awarded subject to the condition that you obtain **full marks** in **at least 8** of the attempted questions.
3. The questions are **self contained**, no doubts will be entertained.

**** Questions starts below ****

Question 1.

[]

The **Affine Cipher** operates on an alphabet of size m . It encrypts a letter represented numerically (x) as $E(x) = (ax + b) \bmod m$, where a and b form the key, and a must be coprime to m (i.e., $\gcd(a, m) = 1$).

Answer the following questions on the cipher described above.

- (a) (2 marks) Given the key ($a = 5, b = 8$) and a 26-letter alphabet ($A = 0, \dots, Z = 25$), encrypt the word **MATH**.
- (b) (2 marks) To decrypt, we need the modular multiplicative inverse of a , denoted a^{-1} , where $a \cdot a^{-1} \equiv 1 \pmod{m}$. **Find** a^{-1} for $a = 5$ and $m = 26$.
- (c) (1 marks) Write the general decryption function $D(y)$ for the **Affine cipher**.

Solution:

- (a) Key: $a = 5, b = 8$, alphabet size $m = 26$, letters numbered $A = 0, B = 1, \dots, Z = 25$.

Encryption: $E(x) = (5x + 8) \bmod 26$.

1. Convert **MATH** to numbers:

$$M = 12, \quad A = 0, \quad T = 19, \quad H = 7.$$

2. Encrypt each letter:

$$E(12) = (5 \cdot 12 + 8) \bmod 26 = 68 \bmod 26 = 16 \Rightarrow Q,$$

$$E(0) = (5 \cdot 0 + 8) \bmod 26 = 8 \Rightarrow I,$$

$$E(19) = (5 \cdot 19 + 8) \bmod 26 = 103 \bmod 26 = 25 \Rightarrow Z,$$

$$E(7) = (5 \cdot 7 + 8) \bmod 26 = 43 \bmod 26 = 17 \Rightarrow R.$$

So $\text{MATH} \rightarrow \text{QIZR}$.

(b) We need $a^{-1} \pmod{26}$ for $a = 5$, i.e., solve $5 \cdot a^{-1} \equiv 1 \pmod{26}$.

We have

$$\begin{aligned} 5 \cdot 5 &= 25 \pmod{26} \Rightarrow 5 \cdot 5 \equiv -1 \pmod{26} \\ &\Rightarrow -(5 \cdot 5) \equiv 1 \pmod{26} \Rightarrow 5 \cdot (-5) \equiv 1 \pmod{26} \\ &\Rightarrow 5 \cdot 21 \equiv 1 \pmod{26} \end{aligned}$$

Students can use any other method, but there should be some work done to show how the answer has been achieved. Brute force is also allowed.

(c) Encryption: $y \equiv ax + b \pmod{m}$. Then

$$y - b \equiv ax \pmod{m} \Rightarrow a^{-1}(y - b) \equiv x \pmod{m}.$$

So the decryption function is

$$D(y) = a^{-1}(y - b) \pmod{m}.$$

Question 2.

[]

The **One-Time Pad (OTP)** operates as follows: the plaintext, ciphertext, and key are binary strings of length L , the **Encryption** is defined as $C = M \oplus K$ (bitwise XOR) and the **Decryption** is defined as $M = C \oplus K$. The key K is uniformly random, used only once, and is as long as the message. Answer the following questions:

- (a) (3 marks) Show mathematically why the OTP satisfies Shannon's perfect secrecy condition.
- (b) (2 marks) What happens if we reuse a key for two different messages?

Solution:

(a) Given: $M, C, K \in \{0, 1\}^L$

The key K is uniformly random: $\Pr(K = k) = 2^{-L}$ for each k .

Encryption: $C = M \oplus K$,

Decryption: $M = C \oplus K$.

Fix any message $m \in \mathcal{M}$ and any ciphertext $c \in \mathcal{C}$. We compute:

$$\Pr[\text{Enc}(k, m) = c] = \Pr[k \oplus m = c] = \Pr[k = m \oplus c],$$

where the probability is over the uniform choice of $k \in \{0, 1\}^n$.

For any fixed m and c , there is exactly one key $k = m \oplus c$ that satisfies $k \oplus m = c$. Since the key is chosen uniformly from 2^n possibilities,

$$\Pr[k = m \oplus c] = \frac{1}{2^n}.$$

This probability is independent of m . Therefore, for any two messages $m_0, m_1 \in \mathcal{M}$,

$$\Pr[\text{Enc}(k, m_0) = c] = \Pr[\text{Enc}(k, m_1) = c] = \frac{1}{2^n}.$$

This satisfies the second formulation of Shannon's definition, so OTP achieves perfect secrecy.

TA Note: There are other similar methods also, verify the correctness and give marks.

(b) If the same key K is used for two messages M_1 and M_2 , then

$$C_1 = M_1 \oplus K \quad C_2 = M_2 \oplus K$$

Then,

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2.$$

Information gained: (Atleast 2 of the following)

- If Eve knows m_1 (e.g., a standard header or predictable message), she can recover $m_2 = c_2 \oplus c_1 \oplus m_1$.
- Even without knowing either plaintext, the XOR $m_1 \oplus m_2$ reveals patterns: identical bits produce 0, differing bits produce 1.
- For English text or structured data, this can lead to complete message recovery using frequency analysis.

Note: The first part (calculation of $C_1 \oplus C_2$) has 1 marks and the information gained has 1 marks

Question 3.

Consider a 5-bit LFSR with the following recurrence relation: $s_{t+5} = s_{t+2} \oplus s_t$, for $t \geq 0$.

Let the Initial state be $[s_4 s_3 s_2 s_1 s_0] = [1 0 1 0 1]$, where s_4 is the leftmost bit and s_0 is the rightmost bit. **Note** that the *rightmost bit* is outputted as the keystream bit.

Answer the following questions on the LFSR defined above:

(a) (2 marks) Draw the LFSR diagram showing the flip-flops, feedback taps, and XOR gate.

(b) (2 marks) Generate the next 4 output bits (beyond the initial state: $[s_4 s_3 s_2 s_1 s_0] = [1 0 1 0 1]$). Show each shift step clearly.

Now consider the same LFSR but with a different initial state: $[s_4 s_3 s_2 s_1 s_0] = [0 0 0 0 0]$. What sequence does this LFSR produce starting from this initial state?

(c) (1 marks) What is the feedback polynomial of this LFSR?

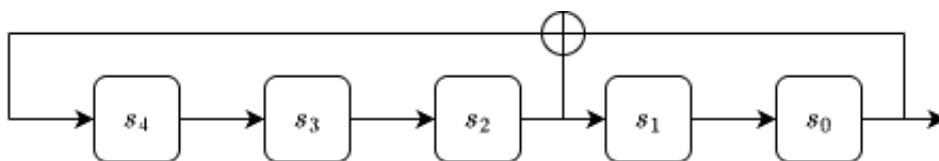


Figure 1: The LFSR

Solution:

(a) The LFSR is as shown in Figure 1

(b) Since the question was to generate the 4 output bits beyond the initial state, the answer should contain the first 5 state changes and then the next 4 output bits, i.e.

total 9 clocks.

Given: Initial state: $[s_4 s_3 s_2 s_1 s_0] = [1 0 1 0 1]$

Step 0 ($t = 0$):

State = $[1 0 1 0 1]$, Output: $s_0 = 1$ Feedback: $F = s_2 \oplus s_0 = 1 \oplus 1 = 0$

New state after shift: $[0 1 0 1 0]$

Step 1 ($t = 1$):

State = $[0 1 0 1 0]$ Output: 0 Feedback: $F = s_2 \oplus s_0 = 0 \oplus 0 = 0$

New state: $[0 0 1 0 1]$

Step 2 ($t = 2$):

State = $[0 0 1 0 1]$ Output: 1 Feedback: $F = 1 \oplus 1 = 0$

New state: $[0 0 0 1 0]$

Step 3 ($t = 3$):

State = $[0 0 0 1 0]$ Output: 0 Feedback: $F = 0 \oplus 0 = 0$

New state: $[0 0 0 0 1]$

Step 4 ($t = 4$):

State = $[0 0 0 0 1]$ Output: 1 Feedback: $F = 0 \oplus 1 = 1$

New state: $[1 0 0 0 0]$

Step 5 ($t = 5$):

State = $[1 0 0 0 0]$ Output: 0 Feedback: $F = s_2(0) \oplus s_0(0) = 0$

New state: $[0 1 0 0 0]$

Step 6 ($t = 6$):

State = $[0 1 0 0 0]$ Output: 0 Feedback: $F = 0 \oplus 0 = 0$

New state: $[0 0 1 0 0]$

Step 7 ($t = 7$):

State = $[0 0 1 0 0]$ Output: 0 Feedback: $F = 1 \oplus 0 = 1$

New state: $[1 0 0 1 0]$

Step 8 ($t = 8$):

State = $[1 0 0 1 0]$ Output: 0 Feedback: $F = 0 \oplus 0 = 0$

New state: $[0 1 0 0 1]$

Outputs for $t = 5$ to $t = 8$: 0, 0, 0, 0

Therefore, the next 4 output bits beyond the first 5 are: 0 0 0 0

All-zero initial state:

Initial state: [0 0 0 0 0]

Since $F = s_2 \oplus s_0 = 0 \oplus 0 = 0$ at every step, the feedback bit is always 0.

Therefore, the register remains in the all-zero state forever, and the output sequence is: 0 0 0 0 ...

(c) The recurrence relations is: $s_{t+5} + s_{t+2} + s_t = 0$.

Substituting $s_{t+k} = x^k$, we get the following characteristic polynomial:

$$C(X) : X^5 + X^2 + X^0 = 0 \implies X^5 + X^2 + 1 = 0$$

Thus the feedback polynomial can be obtained as

$$P(X) = X^5 C(1/X) = 1 + X^3 + X^5$$

Question 4.

[]

Consider a miniature 9-bit version of **Trivium** with the following parameters:

Register A : (a_1, a_2, a_3) , Register B : (b_1, b_2, b_3) and Register C : (c_1, c_2, c_3) . Here, a_1, b_1 , and c_1 are the leftmost positions.

Output rule: $z = a_2 \oplus a_3 \oplus b_2 \oplus b_3 \oplus c_2 \oplus c_3$

Feedback rules:

$$t_A = a_2 \oplus (a_1 \cdot a_2) \oplus a_3 \oplus b_2, \quad t_B = b_2 \oplus (b_1 \cdot b_2) \oplus b_3 \oplus c_2$$

$$t_C = c_2 \oplus (c_1 \cdot c_2) \oplus c_3 \oplus a_2$$

All registers shift to the right, with t_A , t_B , and t_C entering the leftmost positions.

Initial state: $[a_1 \ a_2 \ a_3] = [1 \ 0 \ 1]$, $[b_1 \ b_2 \ b_3] = [0 \ 1 \ 0]$, $[c_1 \ c_2 \ c_3] = [1 \ 1 \ 0]$.

Answer the following questions on the mini **Trivium** defined above:

(a) (4 marks) What is the new register state after one clock cycle.

(b) (1 marks) Compute the output bits z_1 and z_2 .

Solution:

(a) **Initial state:**

$$A = [a_1 \ a_2 \ a_3] = [1 \ 0 \ 1], \quad B = [b_1 \ b_2 \ b_3] = [0 \ 1 \ 0], \quad C = [c_1 \ c_2 \ c_3] = [1 \ 1 \ 0].$$

Clock 1 (before shift):

$$z_1 = a_2 \oplus a_3 \oplus b_2 \oplus b_3 \oplus c_2 \oplus c_3 = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1.$$

$$t_A = a_2 \oplus (a_1 a_2) \oplus a_3 \oplus b_2 = 0 \oplus (1 \cdot 0) \oplus 1 \oplus 1 = 0.$$

$$t_B = b_2 \oplus (b_1 b_2) \oplus b_3 \oplus c_2 = 1 \oplus (0 \cdot 1) \oplus 0 \oplus 1 = 0.$$

$$t_C = c_2 \oplus (c_1 c_2) \oplus c_3 \oplus a_2 = 1 \oplus (1 \cdot 1) \oplus 0 \oplus 0 = 0.$$

Clock 1 (after shift):

$$A = [t_A \ a_1 \ a_2] = [0 \ 1 \ 0], \quad B = [t_B \ b_1 \ b_2] = [0 \ 0 \ 1], \quad C = [t_C \ c_1 \ c_2] = [0 \ 1 \ 1].$$

Thus after Clock 1:

$$A = [010], \quad B = [001], \quad C = [011], \quad z_1 = 1.$$

(b) We already computed z_1 above.

z_2 will be computed from the state after Clock 1 as follows:

$$z_2 = a_2 \oplus a_3 \oplus b_2 \oplus b_3 \oplus c_2 \oplus c_3 = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 0.$$

Final outputs:

$$z_1 = 1, \quad z_2 = 0.$$

Question 5.

[]

Data Encryption Standard (DES) is a 16-round Feistel cipher with 64-bit blocks and 56-bit keys.

You may recall that the round function $F(x, K)$ is defined as:

- *Expansion*: 32-bit $R \rightarrow$ 48-bit $E(x)$ using the expansion permutation.
- *Key mixing*: $A = E(x) \oplus K$, producing a 48-bit result.
- *S-box substitution*: 48-bit $A \rightarrow$ 32-bit B using 8 S-boxes (6 $\rightarrow$ 4 bits each).
- *Permutation*: 32-bit $P(B)$ using a fixed permutation.

Answer the following questions:

- (a) (2 marks) In DES cipher, the decryption algorithm is identical to encryption (with the key schedule reversed)? Prove this mathematically for one round of DES.
- (b) (1 marks) What is the Expansion permutation used in the DES round function described above? Consider that the input bits are numbered $1, 2, 3, \dots, 32$.
- (c) (2 marks) Consider a 32-bit right half R :

$$R = 0xD \ 0x2 \ 0xB \ 0x4 \ 0x6 \ 0x9 \ 0xE \ 0x3$$

Compute the full 48-bit expanded output $E(R)$. Write your answer as eight groups of six bits each.

Solution:

(a) **One round of DES encryption:**

Given input (L_{i-1}, R_{i-1}) , the i -th round produces

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i).$$

One round of DES decryption:

Suppose we are given (L_i, R_i) and want to recover (L_{i-1}, R_{i-1}) . From the encryption equation, $L_i = R_{i-1}$, hence $R_{i-1} = L_i$.

Also, $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$.

Substituting $R_{i-1} = L_i$, we get $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$.

Rearranging, $L_{i-1} = R_i \oplus F(R_{i-1}, K_i)$.

Thus decryption equations are:

$$R_{i-1} = L_i, L_{i-1} = R_i \oplus F(R_{i-1}, K_i).$$

These equations have exactly the same structure as encryption: a swap of halves followed by XOR with the round function.

Note: The answer is correct even if it is shown for the first round, the only thing to check is that the equations are correctly written.

- (b) In DES, the 32-bit input R is expanded to 48 bits by repeating certain boundary bits.

The bits of R are numbered 1 to 32.

Expansion table (DES standard):

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

- (c) **Given:** $R = 0xD\ 0x2\ 0xB\ 0x4\ 0x6\ 0x9\ 0xE\ 0x3$

Thus the 32-bit value of R is:

$$R = 1101\ 0010\ 1011\ 0100\ 0110\ 1001\ 1110\ 0011$$

Step 2: Apply DES Expansion $E(R)$

Using the DES E-box expansion, the 48-bit output is grouped into 8 blocks of 6 bits:

$$E(R) = [111010] [100101] [010110] [101000] [001101] [010011] [111100] [000111]$$

Question 6.

[]

Answer the following questions in $\text{GF}(2^8)$ modulo $m(x) = x^8 + x^4 + x^3 + x + 1$.

- (a) (5 marks) Perform the multiplication of the bytes **0x57** and **0x80**. (Show the reduction step by step using polynomial long division by $m(x) = x^8 + x^4 + x^3 + x + 1$.)
- (b) (1 marks) What is the final result in hexadecimal?

Solution:

(a) Part (a)

1. Convert bytes to polynomials

$$0x57 = 0b01010111 = x^6 + x^4 + x^2 + x + 1$$

$$0x80 = 0b10000000 = x^7$$

2. Multiply $P_1 \times P_2$

$$P_1 = x^6 + x^4 + x^2 + x + 1, \quad P_2 = x^7$$

$$P_1 P_2 = x^7(x^6 + x^4 + x^2 + x + 1) = x^{13} + x^{11} + x^9 + x^8 + x^7$$

Reduce modulo $m(x) = x^8 + x^4 + x^3 + x + 1$.

Let

$$P = x^{13} + x^{11} + x^9 + x^8 + x^7, \quad D = x^8 + x^4 + x^3 + x + 1.$$

Polynomial Long Division in $GF(2)$

Step 1:

$$x^{13} \div x^8 = x^5$$

$$x^5 D = x^{13} + x^9 + x^8 + x^6 + x^5$$

XOR with P :

$$\begin{aligned} & (x^{13} + x^{11} + x^9 + x^8 + x^7) + (x^{13} + x^9 + x^8 + x^6 + x^5) \\ &= x^{11} + x^7 + x^6 + x^5 \end{aligned}$$

Step 2:

$$x^{11} \div x^8 = x^3$$

$$x^3 D = x^{11} + x^7 + x^6 + x^4 + x^3$$

XOR with previous remainder:

$$\begin{aligned} & (x^{11} + x^7 + x^6 + x^5) + (x^{11} + x^7 + x^6 + x^4 + x^3) \\ &= x^5 + x^4 + x^3 \end{aligned}$$

Step 3:

Since $\deg = 5 < 8$, stop.

Final remainder:

$$\boxed{x^5 + x^4 + x^3}$$

Note: This can also be done by substitution, but the answer should be the same.

(b) Convert back to byte

$$x^5 + x^4 + x^3 = 0b\ 00111000 = 0x38$$

Question 7.

[]

AES applies the SubBytes operation to each byte of its 4×4 State matrix using a predefined

S-box. This S-box is constructed using two mathematical steps in $\text{GF}(2^8)$:

- Compute the multiplicative inverse in $\text{GF}(2^8)$ modulo $m(x) = x^8 + x^4 + x^3 + x + 1$, for example, the multiplicative inverse of 0x53 is 0xCA
- Apply an affine transformation over $\text{GF}(2)$.

Answer the following questions:

- (a) (1 marks) What is the complete affine transformation used in the AES S-box construction?

Your answer should include formula for computing each output bit b'_i in terms of the input bits b_i where b'_7, b_7 are the most significant bit and b'_0, b_0 are the least significant bit.

- (b) (4 marks) Using the affine transformation from (a), compute the AES S-box output for the input byte 0x53.

Solution:

(a) AES Affine Transformation

The affine transformation is defined by

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i,$$

for $i = 0, 1, \dots, 7$, where $c = 0x63 \implies c_0 = 1, c_1 = 1, c_2 = 0, c_3 = 0, c_4 = 0, c_5 = 1, c_6 = 1, c_7 = 0$.

The other solution could be in matrix form as follows:

The affine transformation used in the AES S-box is given by

$$b' = Mb + c \pmod{2},$$

where $b = (b_0, b_1, \dots, b_7)^T$ is the input byte (with b_0 the least significant bit), b' is the output byte, M is the 8×8 binary matrix

$$\begin{pmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} \oplus \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \end{pmatrix}$$

where $(c_0 \ c_1 \ c_2 \ c_3 \ c_4 \ c_5 \ c_6 \ c_7) = (1, 1, 0, 0, 0, 1, 1, 0)$.

- (b) Given that the multiplicative inverse of 0x53 is 0xCA,

$$0xCA = 11001010_2$$

Thus

$$b_7 = 1, b_6 = 1, b_5 = 0, b_4 = 0, b_3 = 1, b_2 = 0, b_1 = 1, b_0 = 0.$$

Compute each output bit

$$i = 0 : b'_0 = 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$i = 1 : b'_1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$i = 2 : b'_2 = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$i = 3 : b'_3 = 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$i = 4 : b'_4 = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$i = 5 : b'_5 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$i = 6 : b'_6 = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 1$$

$$i = 7 : b'_7 = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

Output bits (MSB to LSB):

$$b'_7 b'_6 b'_5 b'_4 b'_3 b'_2 b'_1 b'_0 = 11101101$$

$$= 11101101_2$$

Final Result:

$$\boxed{0xED}$$

Question 8.

[]

AES-128 key expansion takes a 16-byte key and generates 44 words (176 bytes) to produce 11 round keys. Answer the following questions on AES key expansion algorithm:

(a) (2 marks) Given $w[0], w[1], w[2], w[3]$, write the formula for computing $w[4]$.

(b) (3 marks) Given

$$w[0] = [0x2B, 0x7E, 0x15, 0x16], \quad w[1] = [0xA0, 0xFA, 0xFE, 0x17]$$

$$w[2] = [0xDB, 0x13, 0x53, 0x45], \quad w[3] = [0x09, 0xCF, 0x4F, 0x3C],$$

compute $w[4]$.

Note: The AES S-box table is provided at the end of the question paper in Figure 3

Solution:

(a) AES-128 Key Expansion

The first word after the initial key is given by

$$w[4] = w[0] \oplus \text{SubWord}(\text{RotWord}(w[3])) \oplus \text{Rcon}[1],$$

where RotWord performs a cyclic left shift of the bytes, SubWord applies the AES S-box to each byte, $\text{Rcon}[1] = 0x01000000$.

TA Note: Even figure with detailed description will be a correct answer.

(b) Given:

$$w[0] = [0x2B, 0x7E, 0x15, 0x16] \quad w[3] = [0x09, 0xCF, 0x4F, 0x3C]$$

Step 1: Apply RotWord

$$\text{RotWord}(w[3]) = [0\text{xCF}, 0\text{x4F}, 0\text{x3C}, 0\text{x09}]$$

Step 2: Apply SubWord (AES S-box)

$$0\text{xCF} \rightarrow 0\text{x8A} \quad 0\text{x4F} \rightarrow 0\text{x84} \quad 0\text{x3C} \rightarrow 0\text{xEB} \quad 0\text{x09} \rightarrow 0\text{x01}$$

Thus,

$$\text{SubWord}(\text{RotWord}(w[3])) = [0\text{x8A}, 0\text{x84}, 0\text{xEB}, 0\text{x01}]$$

Step 3: XOR with $w[0]$

$$[0\text{x2B}, 0\text{x7E}, 0\text{x15}, 0\text{x16}] \oplus [0\text{x8A}, 0\text{x84}, 0\text{xEB}, 0\text{x01}] = [0\text{xA1}, 0\text{xFA}, 0\text{xFE}, 0\text{x17}]$$

Step 4: XOR with Rcon

$$[0\text{xA1}, 0\text{xFA}, 0\text{xFE}, 0\text{x17}] \oplus [0\text{x01}, 0\text{x00}, 0\text{x00}, 0\text{x00}] = [0\text{xA0}, 0\text{xFA}, 0\text{xFE}, 0\text{x17}]$$

Final Result:

$$w[4] = [0\text{xA0}, 0\text{xFA}, 0\text{xFE}, 0\text{x17}]$$

Question 9. []

Consider **CBC** mode with a block size of 8 bits. Let the given plaintext blocks be P_i , ciphertext blocks be C_i , cipher encryption function be $E_K(\cdot)$ and Initialization Vector be IV .

Consider you are given the following:

$$IV = 1010\ 1101, \quad P_1 = 0110\ 1001, \quad P_2 = 1111\ 0000,$$

and the block cipher encryption function is

$$E_K(x) = x \oplus K,$$

i.e., XOR with the key, where $K = 1100\ 1010$.

Answer the following questions based on the CBC mode of encryption described above:

- (a) (1 marks) What is the encryption process of the CBC mode.
- (b) (2 marks) Calculate the ciphertext block C_1 , with the values given above.
- (c) (2 marks) Calculate the ciphertext block C_2 , with the values given above.

Solution:

- (a) The encryption process is

$$C_i = E_K(P_i \oplus C_{i-1}), \text{ with } C_0 = IV \quad (\text{Initialization Vector}).$$

This can also be answered by a diagram

- (b) Given $IV = 10101101$, $P_1 = 01101001$, $K = 11001010$,

First compute:

$$P_1 \oplus IV = 01101001 \oplus 10101101 = 11000100.$$

Now encrypt (since $E_K(x) = x \oplus K$):

$$C_1 = E_K(11000100) = 11000100 \oplus 11001010 = 00001110.$$

Thus, $C_1 = 00001110$.

(c) Given $C_1 = 00001110$, $P_2 = 11110000$,

First compute:

$$P_2 \oplus C_1 = 11110000 \oplus 00001110 = 11111110.$$

Now encrypt:

$$C_2 = E_K(11111110) = 11111110 \oplus 11001010 = 00110100.$$

Thus, $C_2 = 00110100$.

Question 10.

[]

Consider an s -bit OFB mode of operation with block cipher $E_K(\cdot)$ and segment size s bits. Answer the following questions:

- (a) (4 marks) Draw a diagram to show the **encryption of two plaintext blocks** of size s using OFB mode. The diagram should be self-contained, with every component labeled clearly, including: Initialization Vector (IV), Block cipher encryption E_K , Plaintext blocks P_1, P_2 , Ciphertext blocks C_1, C_2 , Feedback path, XOR operations and any other component important to this mode.
- (b) (1 marks) In Output Feedback (OFB) mode, the feedback mechanism uses (*choose the correct option(s)*):
1. The ciphertext output as input for the next block cipher encryption
 2. The plaintext output as input for the next block cipher encryption
 3. The keystream output as input for the next block cipher encryption
 4. The initialization vector repeatedly without any feedback

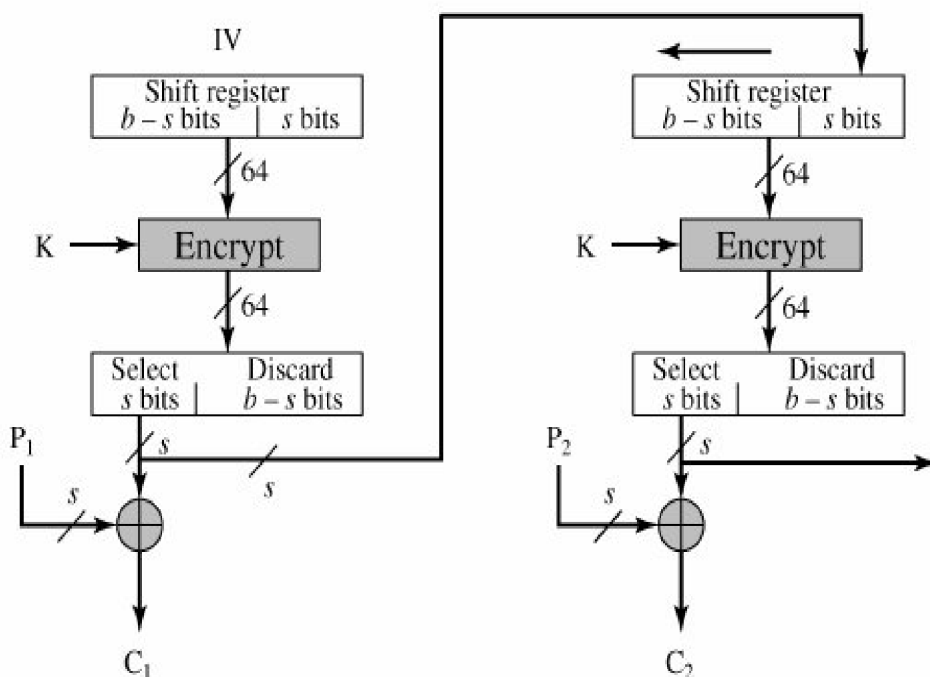


Figure 2: s -bit OFB mode for two plaintexts

Solution:

(a) The diagram is in Figure 2

(b) **Correct:** The keystream output as input for the next block cipher encryption

**** All the best ****

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xa	xb	xc	xd	xe	xf
0x	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1x	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2x	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3x	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4x	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5x	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6x	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7x	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8x	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9x	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
ax	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
bx	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
cx	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
dx	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
ex	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
fx	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Figure 3: AES Sbox